

実験日：2026 年 7 月 3 日

提出期限日：2026 年 7 月 10 日

実際に提出した日：2026 年 7 月 3 日

ナップサック問題

担当教員名： 宮里智樹 教員

学籍番号： 255715B

氏名： 新里仁

課題 1：総当たり法

すべての品物の組み合わせを全探索する手法である。品物数を n とすると計算量は 2^n となり、品物数が増えると指数関数的に計算時間が増大する。

```
items = [
    (4, 6), (8, 12), (3, 4), (5, 3), (9, 7), (2, 1),
    (3, 3), (1, 2), (5, 7), (2, 3), (4, 4), (2, 2),
    (7, 10), (10, 13), (3, 5), (13, 16), (11, 14), (8, 9)
]

capacity = 45
n = len(items)

best_value = 0
best_combination = []
best_weight = 0

for i in range(1 << n):
    current_weight = 0
    current_value = 0
    current_combination = []

    for j in range(n):
        # i の j ビット目が1ならば、品物 j を選ぶ
        if (i >> j) & 1:
            current_weight += items[j][0]
            current_value += items[j][1]
            current_combination.append(j + 1) # 品物番号は1始まりにする

    # 容量の範囲内で、これまでの最大値段を更新した場合
    if current_weight <= capacity:
        if current_value > best_value:
            best_value = current_value
            best_combination = current_combination
            best_weight = current_weight

# 結果の出力
print(f"最大値段: {best_value}")
print(f"品物の組み合わせ (品物番号): {best_combination}")
print(f"合計容量: {best_weight}")
```

課題 2：動的計画法

部分問題の解をテーブルに記憶して再利用する手法である。計算量は nW となり、品物数が多い場合でも高速に解を求めることができる。

```
# 1. 各品物の（容量，値段）のリスト（品物1 ～ 品物18）
items = [
    (4, 6), (8, 12), (3, 4), (5, 3), (9, 7), (2, 1),
    (3, 3), (1, 2), (5, 7), (2, 3), (4, 4), (2, 2),
    (7, 10), (10, 13), (3, 5), (13, 16), (11, 14), (8, 9)
]

capacity = 45 # ナップサックの容量
n = len(items)

# 2. DPテーブルの初期化
# dp[w] は「容量 w のときに達成できる最大の値段」を記録する
dp = [0] * (capacity + 1)

# item_history[w] は「容量 w のときに選んだ品物の番号リスト」を記録する
item_history = [[] for _ in range(capacity + 1)]

# 3. 動的計画法（DP）のループ処理
for idx, (w, v) in enumerate(items):
    item_num = idx + 1 # 品物番号は1始まり

    # ナップサックの最大容量から、今注目している品物の容量 w まで逆順にループ
    # （逆順にする理由は、同じ品物を何回も選んでしまう重複を防ぐため）
    for current_w in range(capacity, w - 1, -1):

        # 「この品物を選んだ場合の値段」が「これまでの最大値段」より高くなる場合
        if dp[current_w - w] + v > dp[current_w]:
            # 値段を更新
            dp[current_w] = dp[current_w - w] + v
            # 選んだ品物の組み合わせ（履歴）を更新
            item_history[current_w] = item_history[current_w - w] + [item_num]

# 4. 結果の抽出
best_value = dp[capacity]
best_combination = item_history[capacity]

# 選択された品物の合計容量を計算
best_weight = sum(items[num - 1][0] for num in best_combination)

# 結果の出力
print(f"最大値段: {best_value}")
print(f"品物の組み合わせ (品物番号): {best_combination}")
```

```
print(f"合計容量:{best_weight}")
```

1 実験結果

VS Code のターミナル上でプログラムを実行した結果、最大値および品物の組み合わせは以下の通りに出力された。詳細な数値を表 1 に示す。本実験の結果、双方のアルゴリズムで同一の最適解が得られることが確認できた。

表 1 実験結果のまとめ（容量 45、品物数 15 の場合）

手法	最大値段（合計価値）	選択された品物番号
総当たり法	64	1, 2, 3, 8, 9, 10, 12, 13, 14, 15
動的計画法	64	1, 2, 3, 8, 9, 10, 12, 13, 14, 15

2 考察

実験結果から、総当たり法と動的計画法のどちらの手法を用いても、得られる最大値段（64）および最適な品物の組み合わせ（[1, 2, 3, 8, 9, 10, 12, 13, 14, 15]）は完全に一致することが確認できた。

しかし、品物の数をさらに増やした場合、総当たり法では指数関数的に探索時間が増大し、実行が困難になった。一方で動的計画法は、テーブルを用いたメモ化により、重複する計算を排除しているため、品物数が増加しても瞬時に解を導き出すことができた。このことから、制約条件が現実的な大きさであるならば、動的計画法を採用することが計算コストの観点から有効であると考察できる。